

To use the string methods in the application:

```
// 1. String Operations
console.log('1. String Operations:')
await redisService.set('user:1', { name: 'John', age: 30 }, 60) // with 60s
TTL

const user = await redisService.get('user:1')
console.log('  Get user:', user)

await redisService.expire('user:1', 120) // Extend TTL
const exists = await redisService.exists('user:1')
console.log('  Exists:', exists)
await redisService.delete('user:1')
console.log('  Deleted user:1\n')
=====
```

To use the hash methods in the application:

```
// 2. Hash Operations
console.log('2. Hash Operations:')
await redisService.hset('profile:1', 'name', 'Alice')
await redisService.hset('profile:1', 'settings', { theme: 'dark',
notifications: true })
const profile = await redisService.hgetall('profile:1')
console.log('  Profile:', profile)
await redisService.hdel('profile:1', 'settings')
console.log('  After delete:', await redisService.hgetall('profile:1'))
console.log('')
=====
```

To use the list methods in the application:

```
// 3. List Operations
console.log('3. List Operations:')
await redisService.rpush('queue:tasks', { id: 1, task: 'email' }, { id: 2,
task: 'sms' })
await redisService.lpush('queue:tasks', { id: 0, task: 'priority' })
const tasks = await redisService.lrange('queue:tasks')
console.log('  All tasks:', tasks)
const firstTask = await redisService.lpop('queue:tasks')
console.log('  Popped:', firstTask)
console.log('  Remaining:', await redisService.lrange('queue:tasks'))
console.log('')
=====
```

To use set methods in the application:

```
console.log('4. Set Operations:')
await redisService.sadd('tags:post:1', 'javascript', 'nodejs', 'redis')
await redisService.sadd('tags:post:1', 'javascript') // Duplicate ignored
const tags = await redisService.smembers('tags:post:1')
console.log('  Tags:', tags)
const hasTag = await redisService.sismember('tags:post:1', 'nodejs')
console.log('  Has nodejs:', hasTag)
await redisService.srem('tags:post:1', 'nodejs')
console.log('  After remove:', await redisService.smembers('tags:post:1'))
console.log('')
=====
// 5. Sorted Set Operations
console.log('5. Sorted Set Operations:')
await redisService.zadd('leaderboard', 100, { user: 'Alice' })
await redisService.zadd('leaderboard', 150, { user: 'Bob' })
await redisService.zadd('leaderboard', 75, { user: 'Charlie' })
const topPlayers = await redisService.zrange('leaderboard', 0, -1, true)
console.log('  Leaderboard (with scores):', topPlayers)
console.log('')
=====
```

To use pipeline and transactions methods:

```
// 6. Pipeline Example (Batch operations)
console.log('6. Pipeline Operations:')
const pipelineResults = await redisService.pipeline((pipe) => {
  pipe.set('pipe:1', JSON.stringify({ data: 1 }))
  pipe.set('pipe:2', JSON.stringify({ data: 2 }))
  pipe.get('pipe:1')
  pipe.get('pipe:2')
  pipe.del('pipe:1', 'pipe:2')
})
console.log('  Pipeline results:', pipelineResults)
console.log('')

// 7. Transaction Example (Atomic operations)
console.log('7. Transaction Operations:')
await redisService.set('balance:alice', 1000)
await redisService.set('balance:bob', 500)

try {
  const txResults = await redisService.transaction((multi) => {
    multi.decrby('balance:alice', 100)
    multi.incrby('balance:bob', 100)
  })
  console.log('  Transaction results:', txResults)
```

```
    console.log('    Alice:', await redisService.get('balance:alice'))
    console.log('    Bob:', await redisService.get('balance:bob'))
  } catch (err) {
    console.error('    Transaction failed:', err)
  }
  console.log('')
  =====
```